

SillyMem

SillyTavern Extension

Requirements Document

Version 1.2

March 2026

Draft

1. Overview

1.1 Purpose

SillyMem is a SillyTavern extension that provides structured, chat-scoped narrative memory for LLM-based roleplay. It detects scene boundaries in chat, generates structured summaries, extracts persistent facts, and maintains a layered memory system that is injected into the LLM's prompt at generation time. Uniquely, its arc system acts as a forward-looking narrative driver — guiding where the story should go next — rather than merely summarizing where the story has been. The extension manages its own storage independently of SillyTavern's built-in lorebook/World Info system. It supports both solo (1:1) chats and group chats.

1.2 Problem Statement

LLMs operating within fixed context windows lose access to earlier conversation history as chats grow. Existing memory solutions for SillyTavern either produce lossy rolling summaries that degrade over time, generate per-message micro-summaries that lose narrative connective tissue, or store memories at the character level rather than scoping them to individual chat sessions. None provide a structured, event-driven memory architecture that mirrors how narrative information is naturally organized: as scenes, story arcs, and persistent world state. Furthermore, no existing solution addresses the problem of narrative drift — long RP conversations that lose direction because the LLM has no concept of where the story is heading, only where it has been.

1.3 Design Philosophy

- **Event-driven, not interval-driven.** Memory operations trigger on narrative boundaries (scene breaks), not on arbitrary message counts or token thresholds.
- **Layered abstraction.** Three tiers of memory (scene summaries, narrative arcs, persistent facts) serve different purposes: scenes record what happened, arcs drive where the story goes next, and facts track the current state of the world.
- **Forward-looking arcs.** Arc directives are prescriptive, not descriptive. They tell the LLM what tensions to develop, what trajectories to follow, and what complications to introduce — providing narrative momentum rather than just a historical record.
- **Chat-scoped.** All memory data is specific to a single chat session. No cross-chat bleed, no shared state between conversations.
- **User-editable at every level.** The user can view, edit, delete, and manually create entries at every memory tier through a dedicated UI panel.
- **Non-invasive storage.** The extension manages its own markdown files and does not write to or depend on SillyTavern's lorebook, World Info, or Data Bank systems.
- **Transparent injection.** The user can see exactly what memory content is being injected into the prompt and can control inclusion/exclusion of individual entries.
- **Context-aware arc generation.** Arc generation draws on all available narrative context — scene summaries, persistent facts, character descriptions, World Info/lorebook content, and persona information — to produce well-informed narrative directives.

1.4 Target Environment

Property	Value
Frontend	SillyTavern (release branch, v1.12+)
Backend	KoboldCPP (primary target); should be compatible with any Text Completion or Chat Completion API SillyTavern supports
Chat Types	Solo (1:1) chats and group chats
Extension Type	Third-party SillyTavern extension (installed via Extensions menu)
Storage	Markdown files managed by the extension, stored in the SillyTavern data directory
Dependencies	No external databases, no additional servers, no Vector Storage requirement

2. Memory Architecture

2.1 Memory Tiers

SillyMem operates on three tiers of memory, each serving a distinct purpose and generated through a different process.

2.1.1 Scene Summaries (Tier 1 — Historical Record)

Scene summaries are the primary memory unit and the authoritative record of what has happened in the story. Each summary captures a single narrative scene: a coherent block of action, dialogue, and events bounded by explicit scene break markers. Scene summaries provide the LLM with a sequential, semi-complete narrative history that maintains consistency across the conversation.

Attribute	Detail
Trigger	Detection of *** scene break delimiter in a chat message
Input	All chat messages from the previous scene break (or chat start) to the current scene break
Output Format	Numbered entry with bolded title, single-paragraph summary (3–6 sentences) covering what happened, who was involved, and narrative significance
Storage	Appended to the memory file under the current arc section
Injection	All scene summaries are injected in sequential order by default; individual scenes can be excluded by the user
Metadata	Scene number, message index range (start–end), generation timestamp
Group Chats	Scene summaries attribute dialogue and actions to the specific characters involved, using their names

2.1.2 Narrative Arcs (Tier 2 — Story Driver)

Narrative arcs are forward-looking directives that guide where the story should go next. Unlike traditional summaries that look backward, arcs are prescriptive: they identify emerging trajectories, escalating tensions, and natural next developments. The LLM uses the active arc as creative direction, giving the story momentum and preventing aimless drift.

The first arc (Arc 1) can be user-written or LLM-assisted. It serves as the initial premise and scenario context — the setup, driving question, and tensions that launch the story. The user may write it entirely by hand, or provide a brief guidance blurb and have the LLM generate a full arc directive from it using the available character, world lore, and persona context. When a subsequent arc's scene threshold is reached, the LLM reviews what actually happened in those scenes and generates a new arc directive for the next phase. All generated arcs are presented to the user as a draft for review and approval before becoming active.

Attribute	Detail
Arc 1 (Initial)	User-written or LLM-assisted. If LLM-assisted, the user provides a brief guidance blurb (a sentence or two of direction) and the LLM expands it into a

	full arc directive using character descriptions, World Info/lorebook content, persona information, and the guidance blurb as input. The result is presented as a draft for approval.
Subsequent Arcs	Generated by the LLM when the scene threshold is reached. The LLM reviews the preceding scenes and writes a forward-looking narrative directive for the next story phase.
Trigger	Automatically generated every N scenes (configurable, default: 2); can also be triggered manually via slash command
Input (Arc Generation)	Scene summaries from the completing arc, current persistent facts, previous arc directive, character description(s), active World Info/lorebook entries, and persona description — providing maximum available context
Output Format	A paragraph-length narrative directive identifying emerging trajectories, escalating tensions, character development opportunities, and suggested complications. Proposes direction without dictating specific outcomes.
Draft Approval	All generated arcs (including LLM-assisted Arc 1) are presented in the Arcs tab as a draft with Accept, Edit, and Regenerate options. The arc does not become active until the user approves it.
Injection	Only the currently active arc is injected. Previous arcs are retained in the file for reference but are not sent to the LLM.
Storage	Written to the memory file as an arc header above its constituent scenes
Group Chats	Arc generation considers all group members' character descriptions as input context. The directive may reference inter-character dynamics and individual character arcs.

2.1.3 Persistent Facts (Tier 3 — World State)

Persistent facts are discrete statements about the current state of the story world. They represent state changes extracted from scenes: who has what, who knows what, what's been destroyed or created, relationship statuses, injuries, locations, and other durable truths that should remain accessible until explicitly contradicted or removed.

Attribute	Detail
Trigger	Extracted during the scene break pipeline, after the scene summary is generated
Input	The scene's raw chat messages or the just-generated scene summary (configurable)
Output Format	Pipe-delimited structured line: FACT subject state keywords
Storage	Stored in the Persistent Facts section of the memory file
Injection	All persistent facts are injected every generation at a configurable depth. No keyword matching or filtering — the full curated facts list is always present.

Lifecycle	Persist until the user manually edits, deletes, or replaces them. New facts that conflict with existing facts (same subject) are flagged in the UI panel for user resolution.
Group Chats	Facts may reference any group member by name. The subject field should identify which character(s) the fact pertains to.

3. Scene Break Pipeline

3.1 Trigger Detection

The extension monitors incoming chat messages (both user and AI-generated) for the presence of the scene break delimiter: a line containing only `**\*\***` (three asterisks), optionally surrounded by whitespace or blank lines. This is the conventional scene/section break marker in roleplay writing.

When a scene break is detected, the extension initiates the scene break pipeline. The delimiter line itself is not included in either the preceding or following scene's content. In group chats, messages from all participants are included in the scene's content regardless of which character generated the scene break.

3.2 Pipeline Sequence

The scene break pipeline executes the following steps sequentially. Each step is a separate LLM call with its own tailored prompt. If any step fails, the extension logs the error, notifies the user via the panel, and continues to the next step. A failed step never blocks subsequent steps or corrupts existing data.

1. **Scene Summary Generation** — The extension collects all chat messages between the previous scene break (or chat start) and the current scene break. These messages are sent to the LLM with the scene summary prompt. The response is validated (must contain a bolded title and at least one sentence of summary), then appended to the memory file under the current arc.
2. **Fact Extraction** — The same scene messages (or the scene summary from Step 1, configurable) are sent to the LLM with the fact extraction prompt template. The response is parsed line-by-line; only lines matching the `FACT | subject | state | keywords` format are accepted. Valid facts are added to the Persistent Facts section. Facts whose subject matches an existing fact's subject are flagged as potential conflicts in the UI panel.
3. **Arc Generation (conditional)** — If the newly completed scene brings the current arc's scene count to the configured arc threshold (default: 2 scenes per arc), the extension generates a new narrative arc. The LLM receives: the completing arc's scene summaries, the current persistent facts, the previous arc directive, the active character description(s), any active World Info/lorebook entries, and the user's persona description. The resulting arc directive is saved as a **draft** in the Arcs tab. It does not become active until the user reviews and accepts it. A new arc section is then started in the memory file for subsequent scenes.

3.3 Pipeline Status Feedback

During pipeline execution, the UI panel displays a progress indicator showing which step is currently running (e.g., "Step 1/3: Generating scene summary..." or "Step 2/3: Extracting facts..."). On completion, the panel updates to reflect new entries. Any failed steps are shown as warnings with the option to retry individually.

3.4 Arc Draft Notification

When a new arc draft is generated (Step 3), the extension displays a prominent notification in the panel and optionally as a toast in the chat UI. The notification indicates that a new arc directive is awaiting review. The chat continues to use the previous arc for injection until the user explicitly accepts the new draft.

4. Prompt Templates

4.1 Template Design Principles

- Each pipeline step uses a dedicated, user-editable prompt template.
- Templates use a rigid output format specification with few-shot examples to maximize structured output reliability from local models.
- Templates are stored as configurable text within the extension's settings, with sensible defaults.
- Templates support SillyTavern macros ({{char}}, {{user}}, etc.) which are resolved at runtime.

4.2 Scene Summary Prompt

The scene summary prompt instructs the LLM to produce a structured scene summary entry from the raw chat messages of a completed scene. The output is a single numbered entry with a bolded event title followed by a paragraph of 3–6 sentences. The summary must cover: what happened during the scene, which characters were involved and what they did, and why the events matter to the ongoing story (cause and effect, character development, relationship shifts, reveals, or stakes changes). Minor back-and-forth dialogue should be collapsed into the broader event rather than itemized.

The prompt includes the previous scene summary (if any) to provide continuity context, and instructs the LLM to maintain strict chronological order and use the characters' names as they appear in the roleplay.

Default scene summary prompt template:

```
Summarize the following roleplay scene as a single structured entry.
```

```
Output format (follow exactly):
```

```
**N. [Short Event Title]** – [A single paragraph of 3-6 sentences summarizing what happened, who was involved, and why it matters to the ongoing story. Cover cause/effect, character development, relationship shifts, reveals, or stakes changes. Collapse minor dialogue into broader events.]
```

```
Rules:
```

- N is the scene number (provided below).
- Use character names exactly as they appear in the scene.
- Stay neutral and descriptive. Do not editorialize.
- If the scene contains mature content, summarize events factually without graphic detail.
- Output ONLY the formatted entry. No preamble, no commentary.

```
Scene number: {{sceneNumber}}
```

```
Previous scene summary (for continuity): {{previousScene}}
```

```
Scene messages to summarize:
```

```
{{sceneMessages}}
```

4.3 Fact Extraction Prompt

The fact extraction prompt uses a rigid template with explicit format rules and few-shot examples:

```
Extract any new or changed facts from this scene.
Output ONLY lines matching this exact format, one per line.
Output NOTHING else – no commentary, no numbering, no markdown.
```

```
FACT | subject | state description | keyword1, keyword2, keyword3
```

Examples:

```
FACT | Sable | now carries the Warden's sigil ring | sable, sigil, ring, warden
FACT | Bridge at Thornhollow | destroyed and impassable | thornhollow, bridge
FACT | Sable, Brennick | reconciled but trust is fragile | sable, brennick, trust
```

Rules:

- One fact per line.
- Subject is WHO or WHAT the fact is about.
- State is the CURRENT condition, possession, knowledge, or relationship.
- Keywords are lowercase, comma-separated terms.
- Only extract facts that represent a CHANGE or NEW information.
- Do not restate facts that have not changed.

The extension's parser accepts only lines beginning with "FACT |" and containing exactly four pipe-delimited segments. All other output is silently discarded.

4.4 Arc Generation Prompt

The arc generation prompt receives the completing arc's scene summaries, the current persistent facts sheet, the previous arc directive, and all available narrative context: character description(s), active World Info/lorebook entries, and persona description. In group chats, all group members' character descriptions are included. It produces a forward-looking narrative directive for the next story phase.

The prompt includes explicit guardrails and a worked example:

```
You are a narrative director for an ongoing roleplay story.
Based on the scenes that have played out, the current world state,
and the characters involved, write a narrative arc directive for the
NEXT phase of the story.
```

Your directive should:

- Identify 2-3 emerging trajectories or tensions that are building
- Suggest complications, developments, or turning points that could unfold
- Note character relationships or dynamics that are ripe for development
- Propose a general direction without dictating specific outcomes
- Draw on the characters' established traits, goals, and secrets

Your directive should NOT:

- Resolve existing tensions – escalate them
- Dictate specific dialogue, actions, or plot points
- Contradict established facts
- Introduce elements completely disconnected from established story threads

```
Write 1-2 paragraphs of narrative direction. This will be used to guide
the story forward, not to summarize what has already happened.
```

Example output:

The uneasy truce between Sable and Brennick is approaching a breaking point – Brennick has noticed the scar moving but hasn't confronted her, and the longer he stays silent the more it will feel like a betrayal of trust when it finally surfaces. The destroyed bridge forces them into the deeper Greenmarch where the corruption is strongest, and Sable's scar will become harder to conceal as it reacts to proximity. Meanwhile, the stolen elven text contains a ritual that could slow the spread – but it requires a second participant who must know what the scar is. The pressure to confide in Brennick should build naturally as the terrain grows more dangerous and Sable's physical symptoms become undeniable.

The same prompt structure is used for LLM-assisted Arc 1 generation, with the scene summaries replaced by the user's guidance blurb. In this case the prompt emphasizes establishing the scenario premise, initial tensions, and character dynamics rather than building on prior scenes.

The output is saved as a draft and presented to the user for approval, editing, or regeneration before it becomes the active arc directive.

4.5 Arc 1 Generation Context

When the user requests LLM-assisted generation of the initial arc, the extension assembles the following context for the prompt:

- **User guidance blurb** — A brief (1–3 sentence) description of the desired scenario direction, provided by the user in the Arcs tab.
- **Character description(s)** — The active character's description field. In group chats, all group members' descriptions are included.
- **World Info / Lorebook** — Any active (constant) World Info entries and any entries whose keywords match terms in the guidance blurb or character descriptions.
- **Persona description** — The user's active persona description, if set.

This context-rich input allows the LLM to generate an initial arc that is grounded in the established characters, world, and the user's intended direction. The result is presented as a draft in the Arcs tab for review.

5. Storage

5.1 File Format

All memory data for a chat is stored in a single markdown file. Markdown is chosen for human readability, ease of manual editing, and trivial parsing. The extension is the sole reader and writer of this file during normal operation, but the user can open and edit it externally at any time.

5.2 File Location and Naming

Memory files are stored in a dedicated subdirectory within SillyTavern's data directory. The exact path follows SillyTavern's extension data conventions. Each file is named to uniquely identify the character (or group) and chat session:

```
<SillyTavern data>/extensions/story-memory/
  {character_or_group_name}_{chat_id}.md
```

For group chats, the group name is used in place of the character name. The chat_id is derived from SillyTavern's internal chat identifier, ensuring uniqueness.

5.3 File Structure

The memory file uses a fixed section layout parsed by the extension via heading markers:

```
# SillyMem - {character_or_group} - {chat_id}

## Persistent Facts
FACT | subject | state | keywords
FACT | subject | state | keywords
...

## Arc 1: [Arc Title]
**Directive:** [Initial premise and story direction]

### Scene 1: [Scene Title] (messages N-M)
**1. [Event Title]** - [Scene summary paragraph]

### Scene 2: [Scene Title] (messages N-M)
**2. [Event Title]** - [Scene summary paragraph]

## Arc 2: [Arc Title] [DRAFT]
**Directive:** [LLM-generated narrative direction for next phase]

### Scene 3: [Scene Title] (messages N-M)
**3. [Event Title]** - [Scene summary paragraph]

...
```

The [DRAFT] marker on an arc heading indicates the arc has been generated but not yet approved by the user. Once approved, the marker is removed and the arc becomes active for injection. Only one arc may be in draft status at a time.

5.4 Excluded Scenes

When the user excludes a scene summary from injection, the extension appends an [EXCLUDED] marker to the scene's heading line:

```
### Scene 3: The Thornhollow Bridge (messages 72-108) [EXCLUDED]
```

The marker is removed when the user re-includes the scene. The scene's content remains in the file regardless of exclusion status — exclusion only affects prompt injection.

5.5 File Integrity

- The extension validates file structure on load. If the file is malformed (e.g., missing section headers), it attempts recovery by re-creating missing sections as empty.
- Write operations use atomic write patterns (write to temp file, then rename) to prevent corruption from interrupted saves.
- The extension never deletes the memory file automatically. File deletion is a manual user action only.

6. Prompt Injection

6.1 Injection Method

SillyMem injects content into the prompt using SillyTavern's extension prompt injection API (setExtensionPrompt). It does not use lorebook entries, Author's Note, or any other built-in SillyTavern prompt feature. All injection positions and roles are configurable. Injection only occurs when the extension is enabled (see §7.5 Global Enable/Disable).

6.2 Injection Layers

Three separate injections occur per generation, each independently configurable. They are listed here in their default injection order (top to bottom in the prompt at the configured depth):

6.2.1 Scene Summary Injection

Setting	Default
Enabled	Yes (toggleable)
Position	In-chat @ Depth
Depth	4
Role	System
Content	All non-excluded scene summaries in sequential order, wrapped in a labeled block
Ordering	Chronological (Scene 1 first, most recent scene last)
Label	Wrapped in: [Story So Far: ...]

6.2.2 Active Arc Injection

Setting	Default
Enabled	Yes (toggleable)
Position	In-chat @ Depth
Depth	4 (same as scene summaries, injected after them)
Role	System
Content	The directive text of the currently active (approved) arc only. Previous arcs and draft arcs are not injected.
Label	Wrapped in: [Current Story Arc: ...]

6.2.3 Persistent Facts Injection

Setting	Default
---------	---------

Enabled	Yes (toggleable)
Position	In-chat @ Depth
Depth	4 (same depth, injected after active arc)
Role	System
Content	The complete persistent facts list. All facts are always injected — no keyword filtering or selective retrieval.
Label	Wrapped in: [Story Facts: ...]

6.3 Injection Ordering

When multiple SillyMem injections share the same depth, they are assembled in this fixed internal order:

4. Scene Summaries (historical record — broadest context)
5. Active Arc Directive (narrative direction)
6. Persistent Facts (world state — most immediate, closest to the generation point)

This ordering places the broadest narrative context first and the most granular, immediately relevant knowledge last. Persistent facts appear closest to the generation point because they represent the current ground truth — who has what, who's where, what's broken — and the LLM gives higher weight to content nearer the bottom of the prompt. The user can adjust depths independently to separate these layers if preferred.

6.4 Token Budget Display

The UI panel displays a running token estimate for the total memory injection payload. This count updates whenever memory content changes or inclusion settings are modified, allowing the user to make informed decisions about what to exclude when context is tight.

7. User Interface

7.1 Panel Location

SillyMem provides a UI panel accessible from SillyTavern's Extensions menu (stacked cubes icon). The panel can optionally be popped out into a floating window for side-by-side monitoring during play.

7.2 Panel Tabs

The panel is organized into three tabs, each corresponding to a memory tier, plus a settings area:

7.2.1 Facts Tab

Element	Description
Fact List	Table view showing all persistent facts: Subject, State, Keywords columns. Each row has inline Edit and Delete buttons.
New Fact Button	Allows manual creation of a new fact entry via input fields for subject, state, and keywords.
Conflict Indicators	Newly extracted facts that share a subject with an existing fact are highlighted with a visual marker (e.g., amber background). The user resolves conflicts by keeping, replacing, or deleting entries.
Search/Filter	Text input to filter the fact list by subject or keyword.
Fact Count	Displays total number of facts and estimated token cost of the full facts injection.

7.2.2 Scenes Tab

Element	Description
Scene List	Sequential list of all scene summaries, each showing: scene number, title, message range, and summary text.
Inclusion Checkboxes	Each scene has a checkbox (checked by default). Unchecking excludes the scene from prompt injection without deleting it.
Inline Edit	Click on any scene summary text to edit it in place.
Delete Button	Removes a scene summary from the memory file entirely.
Manual Add	Button to manually create a scene summary entry (for retroactive scene marking).

7.2.3 Arcs Tab

Element	Description
---------	-------------

Active Arc Display	The currently active arc is displayed prominently at the top, with its directive text and constituent scene titles.
Draft Arc Display	If a draft arc exists, it is displayed with a distinct visual style (e.g., dashed border) and three action buttons: Accept, Edit, and Regenerate.
Accept Button	Approves the draft arc, making it the active arc for injection. Removes the [DRAFT] marker from the file.
Edit Button	Opens the draft arc directive for inline editing before acceptance.
Regenerate Button	Discards the current draft and re-runs the arc generation LLM call to produce a new draft.
Arc History	Previous arcs are listed below in collapsible sections showing their directive and constituent scenes. These are for reference only and are not injected.
Manual Create	Button to manually write and activate a new arc directive at any point, bypassing the automatic generation trigger.
Arc 1 Setup	When no arc exists yet (new chat), the Arcs tab displays a text area for the user to write the initial Arc 1 premise. A “Generate from Blurb” button allows the user to enter a brief guidance blurb (1–3 sentences) and have the LLM generate a full arc directive from it, using character descriptions, World Info, and persona context. The result appears as a draft for review.

7.2.4 Settings

Setting	Description	Default
Extension Enabled	Global toggle to enable or disable SillyMem entirely for the current chat	On
Arc Threshold	Number of scenes per arc before automatic arc generation is triggered	2
Fact Source	Whether fact extraction uses raw scene messages or the generated scene summary as input	Scene summary
Scene Summary Depth	Injection depth for scene summaries	4
Arc Depth	Injection depth for the active arc directive	4
Facts Depth	Injection depth for persistent facts	4
Injection Role	Role assignment for all SillyMem injections (System, User, or Assistant)	System
Scene Summary Prompt	Editable prompt template for scene summary generation	(see §4.2)
Fact Extraction Prompt	Editable prompt template for fact extraction	(see §4.3)
Arc Generation Prompt	Editable prompt template for narrative arc generation	(see §4.4)

Draft Notification	Whether to show a toast notification in the chat UI when a new arc draft is ready	On
--------------------	---	----

7.3 Status Bar

A compact status bar at the top of the panel displays:

- Extension enabled/disabled state.
- Current scene count and arc number.
- Active arc status (active, draft pending, or none).
- Total estimated injection token count.
- Pipeline status (idle, running step N/3, error on step N).

7.4 Popout Mode

The panel supports SillyTavern's standard popout behavior, allowing it to be detached into a floating window. This is intended for users who want to monitor facts and scene summaries alongside the chat in real time. Popout state persists across page reloads.

7.5 Global Enable/Disable

SillyMem can be completely disabled for the current chat via a prominent toggle in the panel's status bar and in the Settings section. When disabled:

- No memory content is injected into the prompt.
- Scene break detection is suspended — *** delimiters are ignored.
- The memory file is not modified.
- All existing memory data is preserved and will be available when the extension is re-enabled.
- The panel remains accessible for viewing and editing memory content, but the status bar clearly indicates the disabled state.

8. Slash Commands

SillyMem registers the following slash commands for power-user and STscript integration:

Command	Description
/sm-enable	Enables SillyMem for the current chat.
/sm-disable	Disables SillyMem for the current chat. No injection, no scene detection.
/sm-scene	Manually triggers the scene break pipeline at the current message, as if *** were detected.
/sm-arc	Manually triggers arc generation for the current arc, regardless of scene count. The result is saved as a draft for review.
/sm-arc-accept	Accepts the current draft arc, making it the active arc for injection.
/sm-arc-set [text]	Manually writes and immediately activates a new arc directive with the given text.
/sm-arc-generate [blurb]	Generates an LLM-assisted arc from the given guidance blurb, using full character/world context. Saved as draft.
/sm-fact [subject] [state] [keywords]	Manually adds a persistent fact entry.
/sm-inject	Displays the current injection payload as a system message for debugging (not sent to the LLM).
/sm-export	Copies the full memory file contents to the clipboard.
/sm-retry [step]	Re-runs a specific pipeline step (scene, facts, arc) for the most recent scene break.

9. Group Chat Behavior

SillyMem is designed to work with both solo (1:1) chats and SillyTavern group chats. The following details describe how the extension adapts to the group chat context:

9.1 Memory File Scoping

Each group chat gets its own memory file, named using the group name and chat ID. Memory data is not shared between the group chat and individual character chats. A character appearing in both a group and a solo chat will have completely independent memory files for each.

9.2 Scene Content

Scene content includes messages from all group participants — all characters and the user. The scene summary prompt receives the full multi-speaker dialogue with character names preserved, so the LLM can attribute actions and dialogue correctly in the summary.

9.3 Arc Generation Context

When generating arcs in a group chat, the extension includes the character descriptions of all group members (not just the active speaker). This gives the LLM awareness of all characters' traits, goals, and dynamics when proposing narrative direction. The World Info context is assembled from all active entries, including any character-specific lorebook entries bound to group members.

9.4 Fact Attribution

Persistent facts in group chats should identify the relevant character(s) in the subject field. The fact extraction prompt is the same for solo and group chats — the LLM naturally attributes facts to specific characters based on the scene content. Users can edit fact subjects to clarify attribution if needed.

10. Error Handling and Edge Cases

10.1 Pipeline Failures

- Each pipeline step is independent. A failure in fact extraction does not prevent arc generation.
- Failed steps log the error, display a warning in the panel's status bar, and offer a per-step retry button.
- If the LLM returns empty or unparseable output for a step, the extension skips that step and notifies the user. No data is written for failed steps.

10.2 Malformed LLM Output

- Scene summaries that lack a bolded title or contain fewer than one sentence are flagged as potentially malformed. The user is prompted to review before the entry is finalized.
- Fact extraction lines that do not match the four-segment pipe format are silently discarded. If zero valid facts are extracted, the panel notes this.
- Arc generation output that appears to summarize past events rather than propose future direction is accepted but noted in the panel. The user can edit or regenerate.

10.3 Scene Break Edge Cases

- Multiple *** delimiters in a single message: treated as a single scene break. The content before the first delimiter belongs to the ending scene.
- *** in the very first message of a chat: ignored (no preceding scene content to summarize).
- Very short scenes (fewer than 2 messages between breaks): the extension still runs the pipeline but includes a note in the panel that the scene may be too short for meaningful summarization.
- User edits/deletes messages after a scene break has been processed: the extension does not retroactively update existing scene summaries. The user can manually edit or re-trigger summarization.

10.4 Arc Edge Cases

- No Arc 1 defined when first scene break occurs: the extension prompts the user to write or generate an initial arc before proceeding, or proceeds without an active arc (scene summaries and facts still function normally).
- User never accepts a draft arc: the previous arc remains active for injection indefinitely. Draft arcs do not expire.
- User manually creates a new arc while a draft is pending: the draft is discarded and the manually created arc becomes active.

10.5 File Recovery

- If the memory file is missing when the extension loads (e.g., manually deleted), the extension creates a fresh file with empty sections.

- If the file exists but has corrupted or missing sections, the extension rebuilds missing sections as empty without discarding intact sections.

11. Future Considerations

The following features are explicitly out of scope for v1.0 but are anticipated as natural extensions of the architecture:

- **Token budget-aware scene injection.** Automatically exclude oldest scene summaries when total injection payload exceeds a configurable token limit. Arc directives and facts would remain unaffected.
- **Cross-chat memory.** Optional mode to share persistent facts (but not scene summaries or arcs) across chats with the same character, enabling a character to “remember” established facts from previous conversations.
- **Automatic scene break detection.** Use heuristics or LLM classification to suggest scene breaks when the user does not manually insert *** delimiters.
- **Export to lorebook.** One-click conversion of the persistent facts sheet into a SillyTavern-compatible lorebook JSON file for users who want to migrate to the native system.
- **Configurable LLM connection.** Allow the user to specify a separate API connection or model for memory operations (e.g., a faster/cheaper model for summarization while the main model handles generation).
- **Arc branching.** Allow the user to save multiple draft arcs and switch between them, enabling exploration of different narrative directions from the same story point.
- **Story State snapshot.** A lightweight, auto-updated current-situation summary derived from the most recent scene summary and active facts, for users who want a quick grounding injection without the full scene history.
- **Per-character fact views in group chats.** Filter the facts panel to show only facts relevant to a specific group member.

12. Glossary

Term	Definition
Scene	A coherent block of narrative action bounded by *** delimiters (or chat start/end).
Scene Break	The *** delimiter that marks the end of one scene and the beginning of the next.
Scene Summary	A structured 3–6 sentence summary of a single scene, generated by the LLM.
Narrative Arc	A forward-looking directive that guides the story’s next phase. Prescriptive, not descriptive.
Arc Directive	The text content of a narrative arc: its trajectory suggestions, tensions, and proposed developments.
Active Arc	The currently approved arc whose directive is injected into the prompt.

Draft Arc	A generated arc directive awaiting user review and approval before becoming active.
Arc 1 / Initial Arc	The first arc, serving as the scenario premise. May be user-written or LLM-assisted from a guidance blurb.
Guidance Blurb	A brief (1–3 sentence) user-written description of desired story direction, used as input for LLM-assisted arc generation.
Persistent Fact	A discrete statement about the current state of the story world, stored with subject and state fields.
Scene Break Pipeline	The sequential three-step process (scene summary, fact extraction, conditional arc generation) triggered by a scene break.
Memory File	The markdown file that stores all memory data for a single chat session.
Injection	The process of inserting memory content into the LLM's prompt at generation time.
Exclusion	User-controlled removal of a specific scene summary from prompt injection (without deleting it from storage).
Conflict Flag	A visual indicator that a newly extracted fact shares a subject with an existing fact, requiring user resolution.